MCMP117 · Week 1 · Course Notes

Fundamentals of Operating Systems

The Big Picture & a History of Computing

What is a computer? What is an operating system? And how did we get here?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Describe a computer as a machine that follows instructions, using the *input* → *process* → *output* model.
2. Give a first, working definition of an operating system (OS) and explain three things it does.
3. Explain *why* we need an OS, by comparing a computer with and without one.
4. Place the major milestones of computing history on a timeline, from Babbage's Analytical Engine to cloud computing.
5. Explain, with a simple calculation, why ideas such as *batch processing*, *multiprogramming* and *time-sharing* were invented.

Contents

1 What Is a Computer, Really? 1

2 What Is an Operating System? 2

3 Why Do We Need an Operating System? 3

4 How Did We Get Here? A History of Operating Systems 3

4.1 Generation 0: Before electronics (1837–1941) . . . 4
4.2 Generation 1: Vacuum tubes and no operating system (1945–1955) . . . 6
4.3 GM-NAA I/O and batch processing (1956) . . . 6
4.4 IBM System/360: one operating system, many machines (1964) . . . 7
4.5 Multiprogramming: letting the CPU multitask (1960s) . . . 7
4.6 Time-sharing: many users, one machine, at once (1960s) . . . 8
4.7 Multics: the giant that did not ship on time (1965–1969) . . . 9

4.8 UNIX is born (1969–1973) . . . 9
4.9 The personal computer (1977–1995) . . . 9
4.10 Linux is born (1991) . . . 10
4.11 Generation 6: the computer in your pocket (2007–2008) . . . 11
4.12 Generation 7: the cloud (2006–today) . . . 11
4.13 The whole timeline at a glance . . . 12

5 Why This History Matters **13**

6 Summary **13**

7 Glossary **14**

8 Try It Yourself **15**

9 Review Questions **15**

10 Looking Ahead: Week 2 **16**

1 What Is a Computer, Really?

Strip away the screen, the keyboard and the apps. At its core, a **computer** is a machine that *follows instructions*. It takes some *input*, *processes* it according to a list of instructions (a **program**), and produces an *output*.

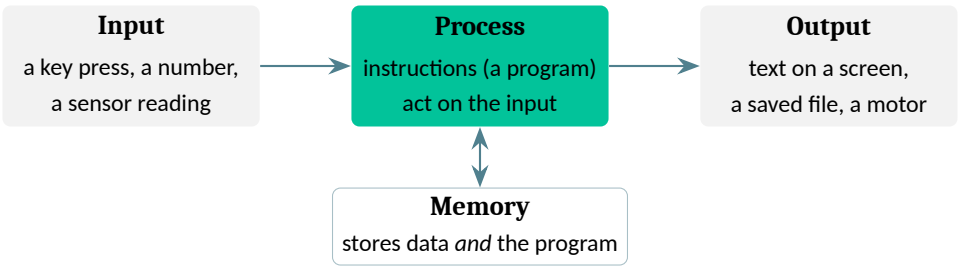


Figure 1: The input–process–output (IPO) model. Memory holds both the data and the program itself.

Two details in Figure 1 matter for the rest of the course:

- **The program is stored in memory, just like data.** This is called the **stored-program** idea. It means we can change what the computer does simply by loading a different program, with no rewiring. As we will see in Section 4.2, the very first electronic computers did *not* work this way.
- **The loop never stops.** The processor repeats the same cycle: *fetch* the next instruction from memory, *decode* it, *execute* it. A modern processor does this billions of times per second.

How fast is "billions of times per second"?

A processor running at a clock frequency of $f = 3$ GHz completes one clock cycle in

$$T = \frac{1}{f} = \frac{1}{3 \times 10^9 \text{ s}^{-1}} \approx 0.33 \text{ ns}.$$

In that time, light travels only about $cT \approx (3 \times 10^8 \text{ m/s})(0.33 \times 10^{-9} \text{ s}) \approx 10$ cm. Waiting even 10 ms for a hard disk therefore costs about 3×10^7 clock cycles. Keep this number in mind: most of operating-system history is about *not wasting* those cycles.

Every computer you will ever meet follows this same loop, whether it is a phone, a laptop, a drone's flight controller or a washing machine.

2 What Is an Operating System?

A first definition (we will sharpen it in Week 2)

The **operating system** is the program that starts first when you turn the computer on, and that manages everything else, so that other programs can run.

This short definition hides three separate jobs:

1. **It starts first.** When you press the power button, a small piece of firmware checks the hardware and then loads the OS. Every other program, whether your browser, a game or a text editor, is started *by* the OS.
2. **It manages everything else.** Many programs want the processor, the memory, the disk and the network at the same time. The OS decides who gets what, and when.
3. **It hides the hard parts.** You press *Save*; you do not need to know which sectors of which disk your file lands on. The OS offers simple, uniform services in place of complicated hardware details.

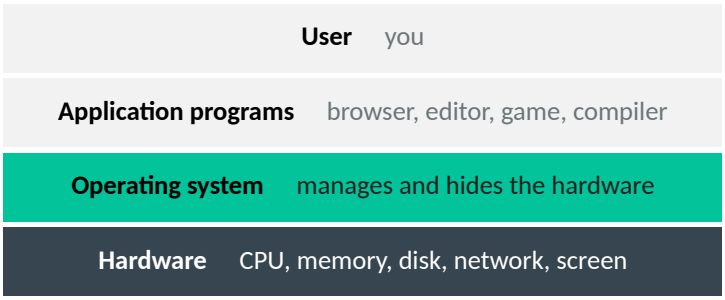


Figure 2: The OS sits between application programs and the hardware.

What happens when you press "Save"?

Suppose you save a 2 MB document in a text editor.

1. The editor asks the OS: *"Write these bytes into a file called* `report.docx`*."* This request is called a **system call** (Week 3).
2. The OS finds free space on the disk, records where the pieces of the file are stored, and updates the folder so the name points to the right place (file systems, Week 7).
3. The OS talks to the disk controller in the controller's own language, waits for it to finish, and tells the editor "done".
4. While the disk is busy, the OS lets *other* programs use the processor (multiprogramming, Section 4.5).

The editor's programmer wrote one line of code for all of this. The OS did the rest.

3 Why Do We Need an Operating System?

The easiest way to see the purpose of an OS is to imagine a computer without one.

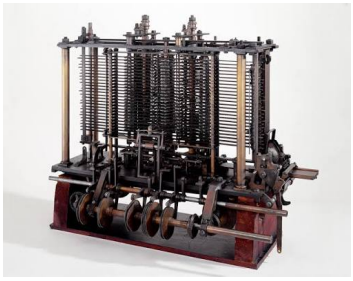
Without an OS	With an OS
Only one program can use the computer at a time.	Many programs safely share the same hardware.
Every program must drive the disk, screen and keyboard itself, which is hard to get right and easy to break.	Programs ask the OS for services; the OS handles the hardware once, correctly.
A buggy program can overwrite another program's memory, or the whole machine.	The OS keeps programs in separate, protected memory areas.
The processor sits idle whenever the program waits for input/output.	The OS gives the processor to another program during the wait.

The left column is not imaginary: it describes almost exactly how the very first computers worked. The rest of these notes tell the story of how the right column was built, one problem at a time.

4 How Did We Get Here? A History of Operating Systems

It is common to divide computing history into *generations*. The boundaries are approximate, but each generation brought a new hardware technology and, with it, a new problem that software had to solve.

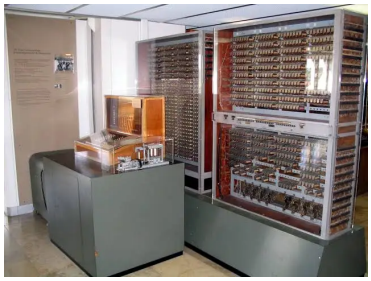
4.1 Generation 0: Before electronics (1837--1941)



(a) Babbage's trial model of the Analytical Engine's mill



(b) Hollerith tabulator with its punch-card reader



(c) Replica of Konrad Zuse's Z3

Figure 3: Programmable machines before electronic computers.

Charles Babbage and the Analytical Engine (1837)

The English mathematician Charles Babbage designed the **Analytical Engine**, a mechanical, general-purpose computer. His design already had the two parts every computer has today: a processing unit, which he called the *mill*, and a memory, which he called the *store*. Instructions and data were to be fed in on punched cards, an idea borrowed from the Jacquard loom, which used punched cards to weave patterns into cloth. The machine was never completed; Figure 3(a) shows a trial model of part of the mill.

In 1843, **Ada Lovelace** published notes on the Analytical Engine that included a step-by-step method for computing Bernoulli numbers. It is widely regarded as the first published computer program. She also saw that such a machine could manipulate any symbols, not only numbers.

Herman Hollerith and the punch card (1890)

The 1880 United States census took roughly eight years to process by hand. For 1890, **Herman Hollerith** built an electromechanical **tabulating machine**. Each person's answers were recorded as holes punched in a card. When a card was pressed into the reader, spring-loaded pins passed through the holes, touched cups of mercury, closed an electric circuit, and advanced a counter dial by one. The 1890 count was finished in a fraction of the previous time.

Hollerith's company later merged with others and, in 1924, became **IBM**. The punch card itself survived for almost a century as the standard way to feed programs and data into computers.

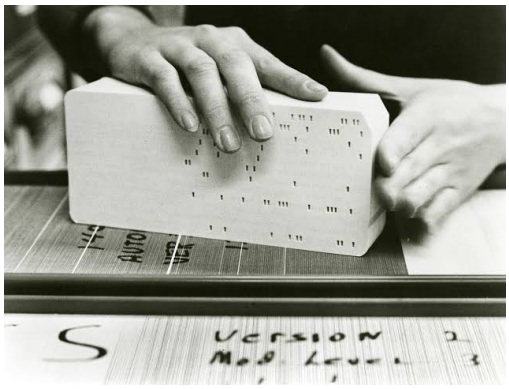


Figure 4: A punch card being read on a card-punch/verifier machine. Each column of holes encodes one character; one card typically held one line of a program.

How much can one card hold?

The standard IBM card had 80 columns and 12 rows of possible hole positions. If we used every position as a binary digit (hole / no hole), one card could hold at most

$$80 \times 12 = 960 \text{ bits} = 120 \text{ bytes}.$$

In practice one column encoded one character, so a card held 80 characters. A program of 2000 lines therefore needed a deck of 2000 cards, a stack about 35 cm high, and dropping it on the floor was a real disaster.

Konrad Zuse and the Z3 (1941)

In Berlin, the engineer **Konrad Zuse** built a series of programmable machines. The **Z3**, completed in 1941 and built from about 2,600 telephone relays, was one of the first working programmable computers. It read its program from punched film. The original was destroyed in a 1943 air raid; the machine in Figure 3(c) is a replica built in the 1960s.

What Generation 0 teaches us

All the ingredients were already there: a processor, a memory, programs on cards. What was missing was *speed*, and there was no software at all to manage the machine.

4.2 Generation 1: Vacuum tubes and no operating system (1945--1955)

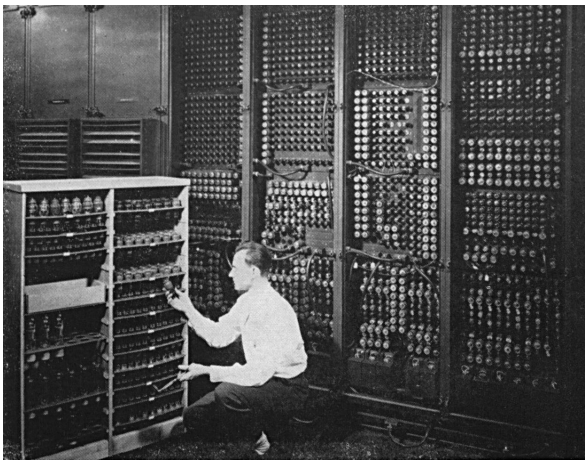


Figure 5: ENIAC (Electronic Numerical Integrator and Computer), 1945.

ENIAC, completed in 1945 at the University of Pennsylvania for the U.S. Army, was one of the first general-purpose *electronic* computers. Some numbers:

- about $18{,}000$ vacuum tubes, about 30 (short) tons, filling a room of roughly 9 m $\times$ 15 m;
- about $5{,}000$ additions per second, thousands of times faster than a human with a desk calculator;
- vacuum tubes failed often, and technicians had to find and replace the broken one.

ENIAC had no keyboard, no screen and **no operating system**. "Programming" meant physically rewiring plugboards and setting thousands of switches, which could take days. The first ENIAC programmers were a team of six women: Kay McNulty, Betty Jennings, Betty Snyder, Marlyn Wescoff, Fran Bilas and Ruth Lichterman.

Only one job ran at a time, with total control of the machine. There was nothing to manage, because there was no other software.

4.3 GM-NAA I/O and batch processing (1956)

By the mid-1950s, computers used transistors, were more reliable, and were very expensive, so every idle minute cost money. Yet after each job finished, a human operator had to walk over, unload the finished job, mount the next job's tapes and cards, and start it. During all that time the computer did nothing.

In 1956, General Motors Research and North American Aviation wrote **GM-NAA I/O** for their IBM 704 mainframe. It kept a small, permanent program, called a **resident monitor**, in memory. Its only job was to load and run the next job automatically, one after another. Because it is the earliest widely cited program built to manage *other* programs, GM-NAA I/O is often called the **first operating system**.

This style of computing is called **batch processing**:

1. **Punch the cards.** A programmer writes the job on a deck of punch cards.
2. **Collect a batch.** An operator collects many jobs and loads them together.
3. **Run automatically.** The monitor runs the jobs one after another, with no human in between.

4. **Collect the printout.** Hours, or a day, later the programmer collects a paper printout and finds out whether there was a typo.

Why the monitor paid for itself

Suppose each job needs $t_{\text{run}} = 2$ minutes of computing, and a human operator needs $t_{\text{setup}} = 8$ minutes to switch between jobs. The fraction of time the processor does useful work, its **utilization**, is

$$U = \frac{t_{\text{run}}}{t_{\text{run}} + t_{\text{setup}}} = \frac{2}{2 + 8} = 0.20 = 20\%.$$

If the resident monitor reduces the switch time to $t_{\text{setup}} = 0.5$ minutes,

$$U = \frac{2}{2 + 0.5} = 0.80 = 80\%.$$

The same machine now does four times as much work per day, with no faster hardware.

4.4 IBM System/360: one operating system, many machines (1964)

Before 1964, each IBM computer model had its own incompatible instruction set, so buying a bigger computer meant rewriting all your software. In 1964 IBM announced **System/360**: a whole family of computers, from small to very large, that all ran the same programs. Its operating system, **OS/360**, was one of the largest software projects ever attempted at that time; it was famously late and over budget. Its project manager, Fred Brooks, later wrote *The Mythical Man-Month* (1975) about the lessons learned.

The key idea: **an operating system can be a product in its own right**, portable across many machines rather than tied to one.

4.5 Multiprogramming: letting the CPU multitask (1960s)

Batch processing removed the human from the loop, but a new waste appeared. A typical job spends much of its time waiting for slow input/output (I/O): reading a tape, a disk or a card. While one job waits, the fast processor sits idle.

The fix is **multiprogramming**: keep *several* jobs in memory at once. When the running job starts waiting for I/O, the OS switches the processor to another job that is ready to compute.

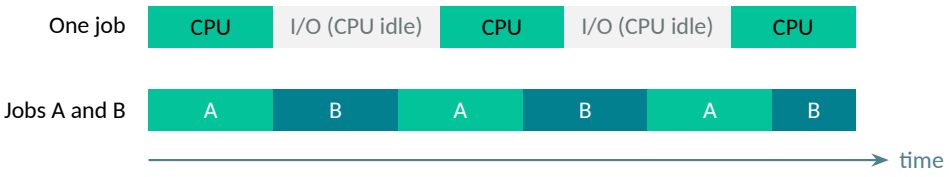


Figure 6: Top: a single job leaves the CPU idle while it waits for I/O. Bottom: with two jobs in memory, the OS fills the gaps.

A simple model of CPU utilization

Suppose each job spends a fraction p of its time waiting for I/O, and assume, for simplicity, that the jobs wait independently of each other. The CPU is idle only when *all* n jobs in memory are waiting at the same time, which happens with probability p^n. Therefore

$$U(n) = 1 - p^n.$$

For a typical I/O-heavy job with $p = 0.8$:

$$U(1) = 1 - 0.8 = 20\%, \qquad U(4) = 1 - 0.8^4 \approx 59\%, \qquad U(10) = 1 - 0.8^{10} \approx 89\%.$$

More jobs in memory means a busier, more productive CPU. The model is simplified, since real jobs do not wait independently, but it shows why multiprogramming was worth the effort, and why *memory* (Week 6) became so important: more memory means more jobs can be resident.

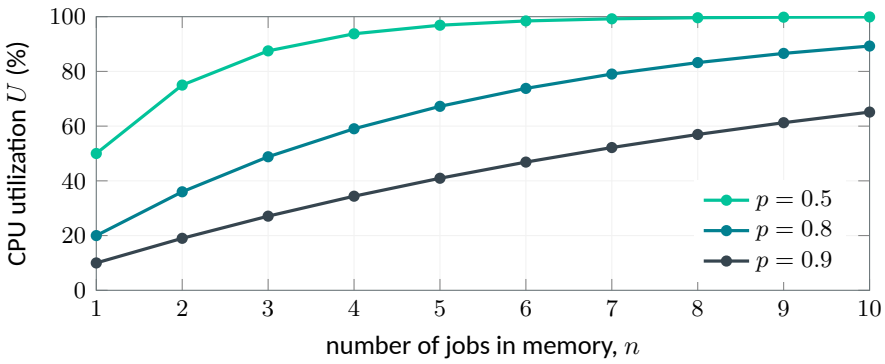


Figure 7: $U(n) = 1 - p^n$ for three different I/O-wait fractions p.

4.6 Time-sharing: many users, one machine, at once (1960s)

Multiprogramming kept the CPU busy, but a programmer still submitted a job and waited hours for the result. **Time-sharing** solved this by connecting many *terminals* (a keyboard plus a printer or screen) to one computer. The OS gives each user a small slice of CPU time, called a **time quantum**, in rapid rotation. Each user feels as if they have the whole machine.

The first successful demonstration was **CTSS** (Compatible Time-Sharing System) at MIT in 1961.

Why it feels instant

Suppose $n = 20$ users share one CPU and the OS gives each a quantum of $q = 10$ ms. In the worst case a user waits for everybody else before running again:

$$T_{\text{wait}} \approx n\,q = 20 \times 10 \text{ ms} = 200 \text{ ms}.$$

A delay of about 0.2 s after pressing a key is barely noticeable to a human. The CPU is shared, but each user experiences an *interactive* computer. With $n = 200$ users, however, the wait grows to about 2 s, which is annoying. Choosing q well is one of the scheduling questions of Weeks 4–5.

Time-sharing is the direct ancestor of every multi-user and multi-process system in use today, including your phone, which time-shares its CPU among dozens of apps and background services.

4.7 Multics: the giant that did not ship on time (1965--1969)

In 1965, MIT, Bell Labs and General Electric started **Multics**, a time-sharing system meant to serve a whole city like a public utility: always on, easy to expand, and secure. Its ideas were far ahead of their time: hierarchical files and directories, dynamic memory management and fine-grained security. But the design was so ambitious that it grew slower and more expensive than planned, and Bell Labs withdrew in 1969. Multics did eventually run, but its most lasting effect was on two Bell Labs researchers who had worked on it.

4.8 UNIX is born (1969--1973)

- **1969.** Ken Thompson writes the first version of **UNIX** on a spare PDP-7 minicomputer at Bell Labs. It is deliberately small and simple, the opposite of Multics.
- **1970–1972.** Thompson and Dennis Ritchie grow it into a set of small, powerful tools that each do one job well and can be chained together.
- **1972–1973.** Ritchie designs the **C** programming language, and UNIX is rewritten in C. This is the single decision that lets UNIX later run on almost any hardware: to move it to a new machine, you mostly re-compile it, instead of re-writing it.

> **The UNIX philosophy (as summarized by Doug McIlroy)**
> Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.

You will use this philosophy directly in Weeks 9–14. For example, the command

```
cat access.log | grep "404" | wc -l
```

chains three tiny programs to count how many lines of a web-server log contain the error code 404.

Because UNIX was written in portable C and, for years, shared with universities, it did not stay one product; it split into a whole family: **BSD** (University of California, Berkeley, which added the TCP/IP networking the early Internet ran on), **HP-UX**, **AIX**, **SunOS/Solaris**, and later Apple's **macOS**, which is a certified UNIX.

4.9 The personal computer (1977--1995)

Apple II (1977)

Designed by Steve Wozniak and sold by Apple Computer, the **Apple II** was one of the first successful mass-produced personal computers. It came with colour graphics and a built-in BASIC language, so it was ready to program the moment it was switched on. For the first time, the person using the computer and the person who owned it were usually the same person.

The IBM PC and MS-DOS (1981)



Figure 8: The IBM Personal Computer (model 5150), 1981.

In 1981 IBM entered the market with the **IBM PC**, built quickly from off-the-shelf parts. Its operating system came from a small company called Microsoft, which had bought an existing system (86-DOS) and adapted it; IBM sold it as PC DOS, and Microsoft sold it to others as **MS-DOS**. MS-DOS was loaded from a floppy disk every time the machine started; a hard drive was not required.

IBM published detailed technical documentation for the PC. Once other companies, starting with Compaq, legally re-created the one piece IBM owned, the start-up firmware called the BIOS, they could build compatible "clone" machines. Thousands did, and "PC compatible" became the industry standard for the next twenty years.

Graphical user interfaces

- **Xerox PARC, 1973.** Researchers build the *Alto*, with overlapping windows, icons and a mouse. It was mostly a research machine and was never sold widely.
- **Apple Macintosh, 1984.** Brings these ideas to a mass-market product: point and click, a trash-can icon, a desktop metaphor, and no typed commands required.
- **Microsoft Windows, 1985–1995.** Starts as a graphical shell on top of MS-DOS; with *Windows 95* it becomes the way most people use a PC.

For most users, typing commands stops being the norm. But the command line never disappears: it is still underneath, and it is where you will work in Weeks 9–14.

4.10 Linux is born (1991)

> **25 August 1991, a Usenet post**
>
> "I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones." — Linus Torvalds, a 21-year-old student in Helsinki

Linux is a new *kernel* (the core of an operating system), written from scratch but deliberately compatible with UNIX in behaviour and commands. It contains no UNIX code; it is a "UNIX-like" system.

Two things made it spread:

- **The licence.** From 1992 Linux was released under the **GNU General Public License (GPL)**: anyone may read, change and redistribute the source code, but modified versions must stay open too.
- **GNU.** Since 1983, Richard Stallman's **GNU Project** had been writing free versions of all the standard UNIX tools, except a kernel. Linux supplied the missing kernel. Together, GNU + Linux formed a complete, free operating system.

Meanwhile, in the server room

While Windows won the desktop, UNIX and Linux won the machines most people never see. They were built from the start for many simultaneous users and long uninterrupted uptime, and Linux is free to license, a decisive advantage once server farms grew to thousands of machines. Today Linux runs most web servers and cloud virtual machines, and since 2017 *all* of the world's 500 fastest supercomputers (the TOP500 list).

4.11 Generation 6: the computer in your pocket (2007--2008)

- **iPhone (2007).** Apple's phone ran a full, modified version of its desktop operating system (what became iOS), not a stripped-down phone OS. It had a capacitive multi-touch screen for fingers, a real web browser, and, from 2008, an App Store. It proved that a modern multitasking OS could run on a battery-powered device.
- **Android (2008).** Android Inc. was founded in 2003 and bought by Google in 2005; the first Android phone (HTC Dream) shipped in 2008. Android's core is a *modified Linux kernel*, and Google licenses it to any phone manufacturer for free. As a result, most smartphones on Earth are, underneath the app icons, running Linux.

4.12 Generation 7: the cloud (2006--today)

Server hardware kept getting more powerful, but most programs did not need a whole physical machine to themselves.

- **Virtualization.** A program called a *hypervisor* lets one physical server pretend to be several independent computers (*virtual machines*), each running its own guest operating system, isolated from the others.
- **Cloud computing.** From 2006, Amazon Web Services rented out virtual machines by the hour: you use someone else's data centre as if it were your own computer.
- **Containers.** An even lighter idea: instead of a whole guest OS, a container shares the host's Linux kernel and packages only the application and what it needs. Platforms such as Red Hat *OpenShift* (2011) and Google's *Kubernetes* (2014) run and manage thousands of containers across many machines.

Notice the pattern: a container is, at its core, a set of features of the Linux kernel that isolate processes from each other. Processes, memory protection and file systems, all topics of this course, are what the modern cloud is built on.

4.13 The whole timeline at a glance

Year	Milestone	Why it matters for operating systems
1837	Analytical Engine (Babbage)	Processor (“mill”) + memory (“store”) + programs on cards
1843	Lovelace’s notes	First published computer program
1890	Hollerith tabulator	Punch cards become the input medium for a century
1941	Zuse Z3	One of the first working programmable computers
1945	ENIAC	Electronic, but no OS: one job, programmed by rewiring
1956	GM-NAA I/O	Resident monitor, batch processing: the first OS
1961	CTSS (MIT)	First time-sharing system: interactive computing
1964	IBM System/360, OS/360	One OS for a whole family of machines
1965–69	Multics	Ambitious time-sharing; inspires UNIX
1969	UNIX (Bell Labs)	Small, simple, composable tools
1973	UNIX rewritten in C	Portable OS: runs on almost any hardware
1977	Apple II	Personal computer for homes and schools
1981	IBM PC, MS-DOS	PC-compatible standard; OS loaded from floppy
1983	GNU Project	Free-software versions of UNIX tools
1984	Apple Macintosh	Graphical interface for the mass market
1991	Linux	Free UNIX-like kernel; with GNU, a complete free OS
1995	Windows 95	GUI becomes the default PC experience
2006	Amazon Web Services	Cloud computing: rent virtual machines by the hour
2007	iPhone	Full multitasking OS in a phone
2008	Android	Linux kernel goes into billions of pockets
2011 / 2014	OpenShift / Kubernetes	Containers managed at scale on Linux

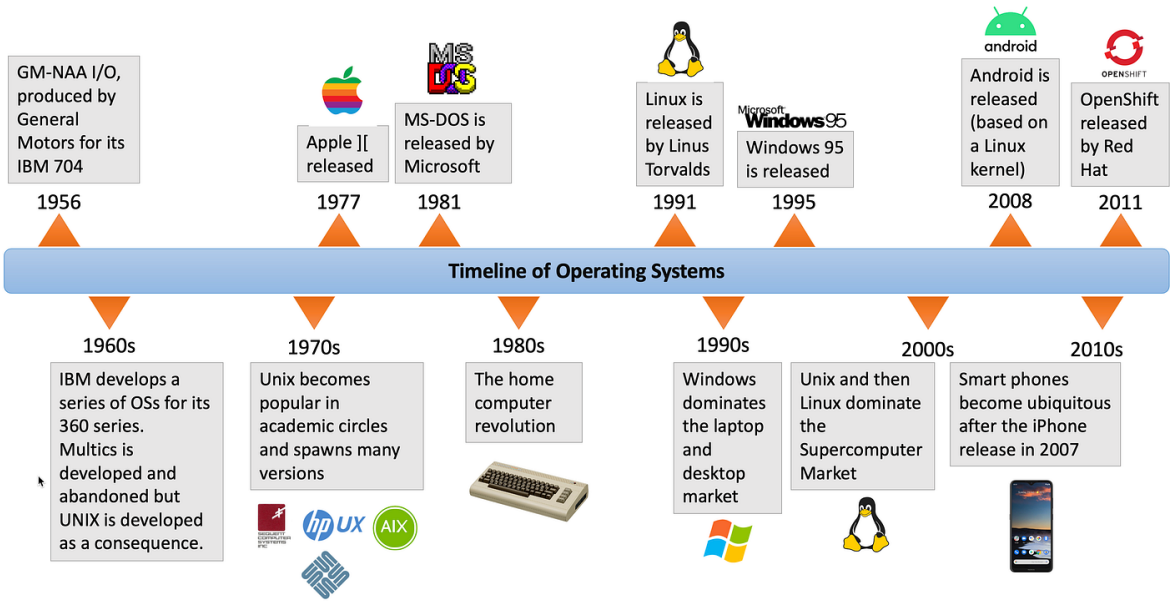


Figure 9: Timeline of operating systems, from GM-NAA I/O (1956) to OpenShift (2011).

5 Why This History Matters

1. **Every generation solved a real problem.** No OS → automation (batch) → sharing the CPU (multiprogramming) → sharing it interactively (time-sharing) → sharing it across the world (cloud). Every OS feature exists because someone was losing time or money without it.
2. **Old ideas never really disappear.** Your phone still multiprograms, still time-shares, and still runs "batch" jobs in the background (backups, updates).
3. **This course follows the same order.** We study processes, scheduling, memory and files roughly in the order they were invented to solve these problems.

Historical idea	Where you meet it again in MCMP117
Batch jobs waiting their turn	Weeks 3–4: process concept, process states
Multiprogramming and time-sharing	Weeks 4–5: CPU scheduling (FCFS, SJF, Priority, Round Robin)
Multics' memory and security ideas	Weeks 6–7: memory management, paging, virtual memory
Hierarchical files and directories	Week 7: file systems and directory structures
UNIX's small, composable tools	Weeks 9–14: Linux and Bash, full practical track

6 Summary

Week 1 in six sentences

1. A computer, at its core, follows instructions: input → process → output, with the program stored in memory.
2. A first working definition: the OS is the program that starts first and manages everything else.
3. We need an OS so that programs can safely share hardware, and so that programmers and users do not have to manage the hardware themselves.
4. History moved from no software (ENIAC) → batch → multiprogramming/time-sharing → personal computers → mobile and cloud.
5. Each step was driven by the cost of an idle processor, which we can estimate with simple formulas such as $U = \dfrac{t_{\text{run}}}{t_{\text{run}} + t_{\text{setup}}}$ and $U(n) = 1 - p^n$.
6. Two dates worth remembering: UNIX, 1969; Linux, 1991, the system we will use hands-on later this semester.

7 Glossary

Batch processing
Running a collected group of jobs one after another automatically, with no human interaction between them.

Kernel
The core part of an operating system, always in memory, that controls the hardware and manages resources.

Multiprogramming
Keeping several jobs in memory so the CPU can switch to another job while one waits for I/O.

Operating system (OS)
The program that starts first and manages the computer's hardware and all other programs.

Program
A list of instructions that tells the computer what to do.

Resident monitor
A small program kept permanently in memory that loads and starts jobs automatically; the earliest form of OS.

Stored-program computer
A computer that keeps its program in memory, like data, so it can be changed without rewiring.

System call
A request from a program to the operating system for a service (e.g. "open this file").

Time quantum
The small slice of CPU time the OS gives each program or user in turn.

Time-sharing
Switching the CPU among many interactive users so quickly that each feels they have the whole machine.

Utilization
The fraction of time a resource (usually the CPU) spends doing useful work.

Virtual machine
A software-created computer, running its own OS, on top of a real (physical) machine.

8 Try It Yourself

See multiprogramming on your own computer

Windows: press `Ctrl+Shift+Esc` to open Task Manager and look at the *Processes* tab. **Linux/macOS:** open a terminal and type `top` (press q to quit).

1. How many processes are running right now? (Usually more than 100.)
2. How many CPU cores does your computer have? (Task Manager → *Performance*.)
3. If there are far more processes than cores, how can they all be "running"? Write two sentences using the words *time-sharing* and *multiprogramming*.

9 Review Questions

Conceptual

1. Draw the input–process–output model and give one example of each part for a smartphone camera.
2. State the first working definition of an operating system and explain its three jobs in your own words.
3. Give two problems that arise when there is no operating system and several programs want the same computer.
4. Why is ENIAC described as a computer with "no operating system at all"?
5. What was a *resident monitor*, and what problem did it solve?
6. Explain the difference between *multiprogramming* and *time-sharing*. Which one is mainly about CPU efficiency, and which one is mainly about the user's experience?
7. Why was rewriting UNIX in C in 1973 such an important decision?
8. Linux contains no UNIX code. In what sense is it still a member of the UNIX family?
9. Give one reason Linux won in servers even though Windows won on the desktop.
10. How is a container different from a virtual machine?

Numerical

11. A job runs for $t_{\text{run}} = 3$ min and the operator needs $t_{\text{setup}} = 9$ min between jobs. Compute the CPU utilization. What must t_{setup} become for $U = 90\%$?
12. Jobs spend $p = 0.7$ of their time waiting for I/O. Using $U(n) = 1 - p^n$, find U for $n = 1, 2, 5$. What is the smallest n that gives $U \geq 95\%$?
13. A time-sharing system has n users and a quantum $q = 20$ ms. Users complain if they wait more than 0.5 s. What is the largest number of users the system can support?
14. A processor runs at 2.5 GHz. How many clock cycles pass while it waits 5 ms for a disk?

Answers to the numerical questions

11. $U = \dfrac{3}{3+9} = 25\%$. For $U = 0.9$: $\dfrac{3}{3+t_{\text{setup}}} = 0.9 \Rightarrow t_{\text{setup}} = \dfrac{3}{0.9} - 3 \approx 0.33$ min $= 20$ s.

12. $U(1) = 30\%$, $U(2) = 1 - 0.49 = 51\%$, $U(5) = 1 - 0.7^5 \approx 83\%$. We need $0.7^n \le 0.05$, i.e. $n \ge \dfrac{\ln 0.05}{\ln 0.7} \approx 8.4$, so $n = 9$.

13. $n\,q \le 0.5$ s $\Rightarrow n \le \dfrac{0.5}{0.020} = 25$ users. (Real systems also lose time switching between users, so the true limit is a little lower.)

14. $N = f\,t = (2.5 \times 10^9\ \text{s}^{-1})(5 \times 10^{-3}\ \text{s}) = 1.25 \times 10^7$ cycles.

10 Looking Ahead: Week 2

Next week we replace our informal definition with a precise one, and look inside the operating system:

- a formal definition of an operating system, and its two classic views: *resource manager* and *extended machine*;
- the design goals of an OS: convenience, efficiency and the ability to evolve;
- the four components of a computer system: hardware, operating system, application programs and users;
- how operating systems are built: monolithic, layered, microkernel and modular structures.

Further reading (optional)

- R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, free online textbook: `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, Chapter 1.
- A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, Section 1.2 (“History of Operating Systems”).
- D. M. Ritchie, “The Evolution of the Unix Time-sharing System” (1979): `https://www.bell-labs.com/usr/dmr/www/hist.html`